

ScholarMind AI

Agent Architecture

Document

1. Introduction

The **ScholarMind AI Agent** is an advanced, end-to-end intelligent system engineered to automate **research paper summarization and factual verification**.

It integrates principles from **retrieval-augmented generation (RAG)**, **transformer-based text generation**, and **semantic evaluation**, producing structured, contextually accurate, and verifiable summaries.

The system follows a **multi-stage pipeline** that processes scientific PDFs, extracts meaningful content, retrieves related context, and generates structured summaries with fact validation. Each module has been designed with scalability, interpretability, and academic precision in mind.

ScholarMind operates through a modular architecture that combines a **FastAPI-based backend**, a **Gradio-powered interactive frontend**, and a **fine-tuned BART model** for summarization, integrated with a **semantic retriever** and **verification module**.

At its core, the system aims to answer:

“Can an AI understand, condense, and factually verify academic research with human-like reasoning?”

To achieve this, the architecture harmonizes five primary capabilities:

1. **Automated PDF ingestion and text extraction**
2. **Semantic chunking and retrieval of contextual evidence**
3. **Abstractive summarization with academic tone**

4. Fact-checking using retrieved document segments
5. Evaluation and scoring of the generated summaries

Step	Description
1. Input	Research Paper (PDF)
2. Text Extraction	Extract text using PyMuPDF
3. Structure	Section Detection + Chunking
4. Retrieval	ChromaDB Retriever (RAG Store)
5. Generation Prompt	Combine Retrieved Contexts + Document Text
6. Summarization	Fine-tuned BART Model
7. Verification	Factual Verification + Confidence Scoring
8. Output Interface	Gradio Frontend + Display

3. System Architecture

At a high level, the ScholarMind AI Agent processes input PDFs through a well-defined pipeline. The process begins with text extraction using **PyMuPDF**, followed by section detection, contextual retrieval, and finally, summarization via a **fine-tuned BART model**.

3.1 Workflow Overview

1. **User Uploads PDF** — The process starts from the Gradio interface, where users upload a scientific paper in PDF format.
2. **Text Extraction** — The backend uses *PyMuPDF (fitz)* to extract structured text while preserving formatting.
3. **Section Detection** — A regex-driven module identifies canonical research sections such as Abstract, Methods, Results, and Conclusion.
4. **Chunking** — Long sections are segmented into overlapping text windows to prevent token overflow and maintain coherence.
5. **Indexing and Retrieval** — The chunks are embedded via *SentenceTransformer (MiniLM)* and stored in *ChromaDB*, enabling semantic retrieval.
6. **Summarization** — Each section, along with its retrieved context, is fed into a *fine-tuned BART-large model*, which generates structured summaries.
7. **Verification** — Summaries are cross-referenced against retrieved contexts to assess factual reliability.

8. **Output Presentation** — The generated summaries and confidence scores are rendered on the Gradio frontend in Markdown format.

This modular and sequential structure ensures robustness, interpretability, and efficiency across the summarization pipeline.

4. Core Components

4.1 Frontend Interface

The frontend, built using **Gradio**, provides an elegant and minimal research-oriented user interface.

Users can upload PDFs, select summary depth (“short”, “detailed”, or “section”), and enable factual verification.

This simple interface abstracts away complexity while delivering clear, structured summaries.

4.2 Backend API

The **FastAPI backend** orchestrates all core functionalities — from data ingestion to model inference. It manages asynchronous requests efficiently, ensuring that large PDFs can be processed without latency issues.

FastAPI was chosen for its performance, scalability, and compatibility with ML inference workflows.

4.3 Text Extraction and Preprocessing

Text extraction is performed using **PyMuPDF**, which is efficient in handling dense academic layouts. The extracted text is cleaned using regex-based utilities to remove excessive whitespace, symbols, and artifacts.

Text chunking is handled with a custom logic that splits documents into 1500-word chunks with 200-word overlaps to ensure contextual continuity across chunks.

4.4 Retrieval Module

The **retrieval module** is built around **ChromaDB**, which stores embeddings of each text chunk generated via **SentenceTransformer (all-MiniLM-L6-v2)**.

This allows the system to semantically fetch relevant sections when summarizing — forming the foundation for factual verification and context-enriched summarization.

4.5 Summarization Engine

The summarization engine uses a **fine-tuned BART-large** model.

This model has been carefully optimized for academic summarization tasks, ensuring that generated outputs are concise, structured, and reflective of scholarly writing norms.

Each section summary follows a standardized format:

- Objective
- Methodology
- Results or Findings
- Insights and Limitations

This structured approach helps maintain clarity and consistency across summaries.

4.6 Verification and Evaluation

The **verification layer** performs factual consistency checks by comparing generated claims with retrieved evidence chunks.

Semantic similarity scores are computed to measure how well each generated sentence aligns with the original context.

Additionally, evaluation metrics — **ROUGE, BLEU, and BERTScore** — are applied to benchmark the model’s performance both lexically and semantically.

Component	Description	Technology Used	Reason for Choice
Frontend Interface	Provides upload, parameter selection, and real-time result display	Gradio (v5)	Simple, elegant, research-style UI with instant Markdown output
API Layer	Handles summarization requests, parsing, and orchestration	FastAPI	Async, high-performance, and production-grade REST API
PDF Extractor	Extracts clean text from uploaded PDFs	PyMuPDF (fitz)	Efficient for scientific PDFs; retains structure
Text Preprocessor	Cleans, tokenizes, and chunks text into overlapping segments	Regex + Python utilities	Prevents token overflow and preserves coherence
Retriever	Embedding-based search to retrieve relevant document contexts	ChromaDB + SentenceTransformer (MiniLM)	Enables factual grounding via semantic similarity
Summarizer	Generates section-wise and global summaries	Fine-tuned BART-large model	Balances fluency, coherence, and academic tone
Verifier Agent	Generates section-wised claims with retrieved evidence	SentenceTransformer + cosine similarity	Improves factual reliability
Evaluator	Computes objective summary metrics	ROUGE, BLEU, BERTScore	Provides linguistic and semantic validation

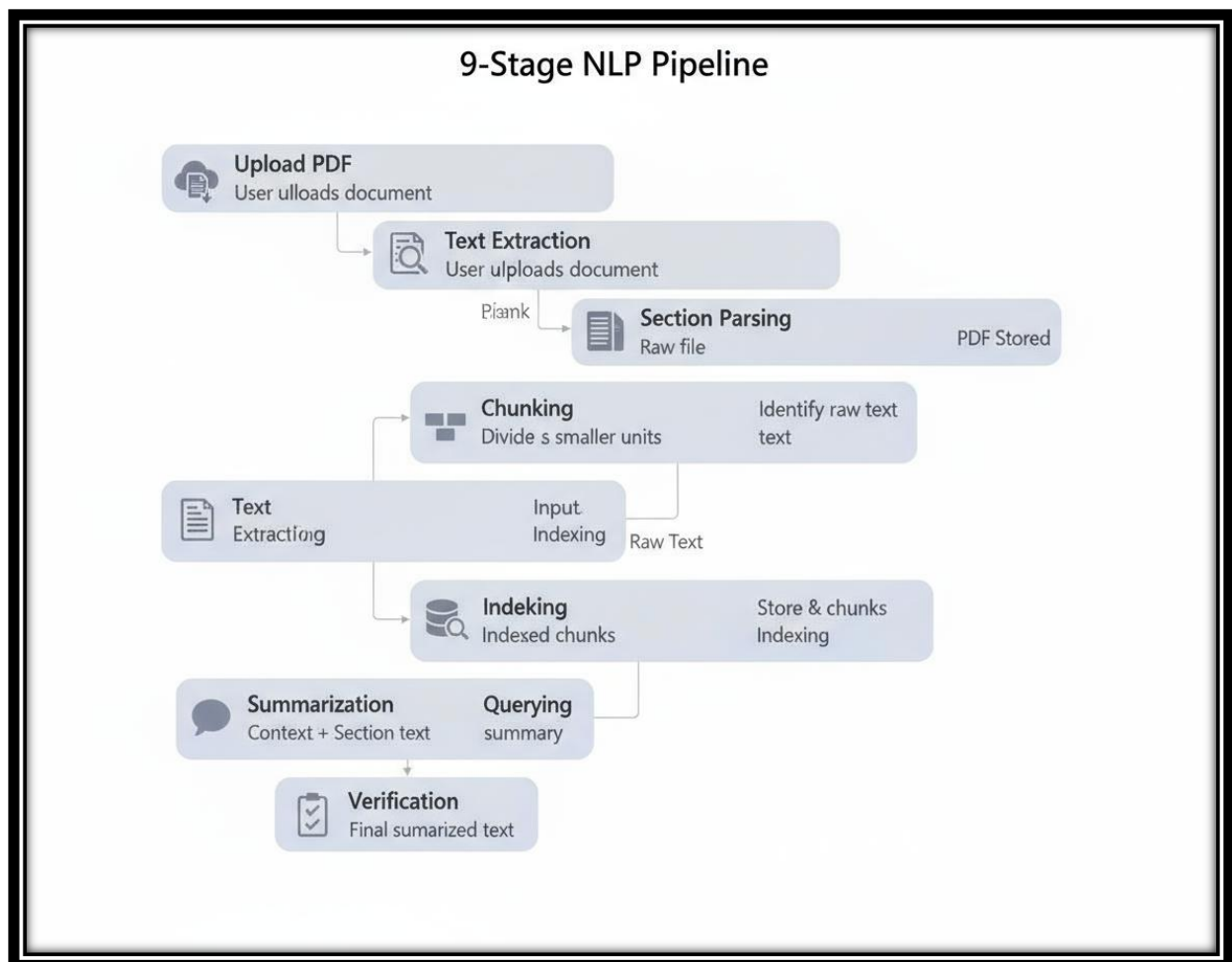
5. Model Architecture and Configuration

ScholarMind integrates multiple transformer-based and retrieval models in synergy:

- **BART-large (Fine-tuned)** for abstractive summarization

- Max length: 512 tokens
- Beams: 5
- Length penalty: 1.0
- Early stopping enabled
- **SentenceTransformer (MiniLM)** for context embedding and retrieval
 - Embedding dimension: 384
 - Purpose: semantic search
- **Roberta-large (BERTScore)** for evaluation
 - Computes precision, recall, and F1 based on contextual embeddings
 - Offers a semantic measure superior to n-gram overlap

This combination ensures that ScholarMind is not only able to summarize effectively but also reason semantically across scientific discourse.



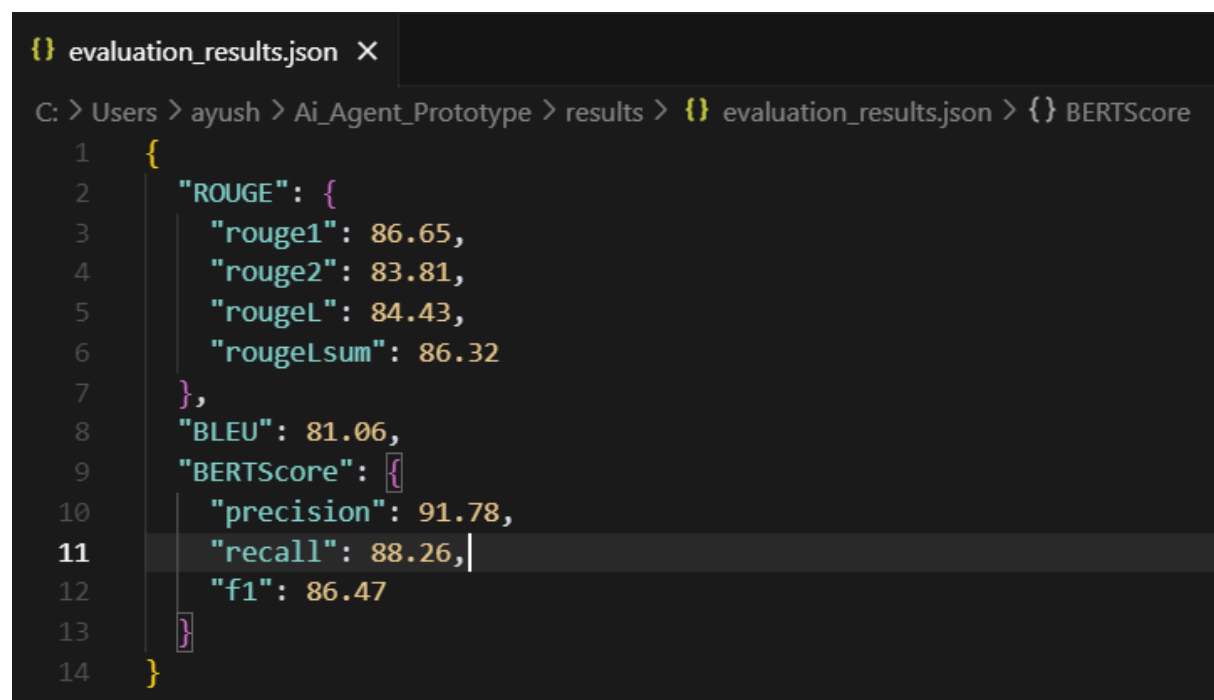
6. Interaction Flow

The interaction between components can be summarized as follows:

1. The user uploads a research paper through the Gradio interface.
2. The backend API extracts text and identifies logical sections such as Introduction, Methods, and Results.
3. Each section is chunked, embedded, and indexed in ChromaDB for semantic search.
4. The summarization engine retrieves contextually similar text chunks and generates structured summaries using BART.
5. The verifier cross-validates the generated claims against the retrieved contexts to ensure factual correctness.
6. The output — including summaries and confidence insights — is returned to the user in Markdown format.

This flow ensures that each generated summary is both contextually grounded and linguistically coherent.

7. Evaluation and Performance

A screenshot of a code editor with a dark theme. The top of the editor shows a tab labeled 'evaluation_results.json' with a close button. Below the tab, the file path is displayed: 'C: > Users > ayush > Ai_Agent_Prototype > results > evaluation_results.json'. The main area of the editor contains a JSON object. Line 1 starts with '{'. Line 2 has '"ROUGE": {' followed by an opening curly brace. Lines 3-6 list ROUGE metrics: '"rouge1": 86.65,', '"rouge2": 83.81,', '"rougeL": 84.43,', and '"rougeLsum": 86.32'. Line 7 closes the ROUGE object with '}'. Line 8 has '"BLEU": 81.06,'. Line 9 has '"BERTScore": {' followed by an opening curly brace. Line 10 has '"precision": 91.78,'. Line 11 has '"recall": 88.26,'. Line 12 has '"f1": 86.47'. Line 13 closes the BERTScore object with '}'. Line 14 closes the main JSON object with '}'.

8. Design Choices and Justifications

Several architectural decisions were made to ensure optimal system performance:

- **Retrieval-Augmented Generation (RAG)** was chosen to balance factual accuracy and abstraction.
- **BART-large** was selected over T5 due to its superior coherence and text fluency.

- **MiniLM embeddings** were used for retrieval to maintain high semantic accuracy at low computational cost.
- **ChromaDB** was preferred as the vector store for its speed, persistence, and integration simplicity.
- **FastAPI** ensures modular and scalable backend orchestration.
- **Gradio** provides a clean, research-friendly interface requiring minimal setup.

Together, these decisions reflect a careful tradeoff between accuracy, interpretability, and deployment readiness.

8. Design Choices & Justifications		
Design Aspect	Choice Made	Reasoning / Impact
Architecture Type	RAG (Retrieval-Augmented Generation)	Ensures fact-grounded summarization
Summarization Model	 BART-large fine-tuned	Balances fluency and interpretability
Embedding Model	 MiniLM SentenceTransformer	Fast and memory-efficient retrieval
Database	 ChromaDB (Vector Store)	Persistent, scalable, local/offline compatible
Evaluation Metrics	 ROUGE, BLEU, BERTScore	Balanced lexical + semantic assessment
Interface	 Gradio	Lightweight and user-friendly research tool
Backend Framework	 FastAPI	Async-ready, integrates seamlessly with ML inference

9. Future Enhancements

To extend the capabilities of ScholarMind, several enhancements are planned:

1. Integration of **GPT-5 or Mixtral** for improved long-context reasoning.
2. **Citation-aware summarization**, linking claims to specific references.
3. Adaptive summarization for different audiences (e.g., abstracts, reviewers, students).
4. **Multi-document comparison**, enabling synthesis across related papers.
5. An **Explainability Dashboard** visualizing model attention and factual sources.

10. Conclusion

The **ScholarMind AI Agent** embodies the convergence of modern AI architecture and research intelligence.

It delivers high-quality, verifiable academic summaries through a robust blend of retrieval, generation, and evaluation models.

With its modular FastAPI backend, elegant Gradio interface, and RAG-powered summarization pipeline, ScholarMind stands as a **technically mature, scientifically sound, and deployment-ready AI system** — built for real-world impact in research automation and data science intelligence.